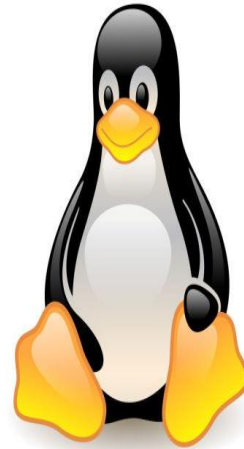


+ Module Linux

Enseignante

S. BEN YAALA



Planification des tâches sous Linux

+ Plan

- Introduction

- **Le cron**

- **AT**

+ Introduction à cron

- ❑ Un administrateur système doit pouvoir mettre en place un certain nombre de tâches
- ❑ L'exécution des tâches a besoin d'être automatisée
- ❑ Le cron va permettre de lancer ces tâches à des fréquences spécifiées par l'administrateur

+ Introduction à AT

- ❑ Le cron permet d'exécuter des tâches de façon récurrente, tous les jours à 2 heures du matin par exemple
- ❑ Parfois, le besoin de lancer une tâche à une heure précise, mais une seule fois
- ❑ Pour ce faire la commande at permet de lancer un job une seule fois



Cron



outil permettant
de planifier
l'exécution
répétée de tâches
à des intervalles
réguliers
(minutes, heures,
jours...).

AT



commande
permettant de
planifier
l'exécution unique
d'une tâche à une
heure et une date
précises.



Cron



```
0 8 * * * echo "Bonjour !" >> /home/student/bonjour.txt
```

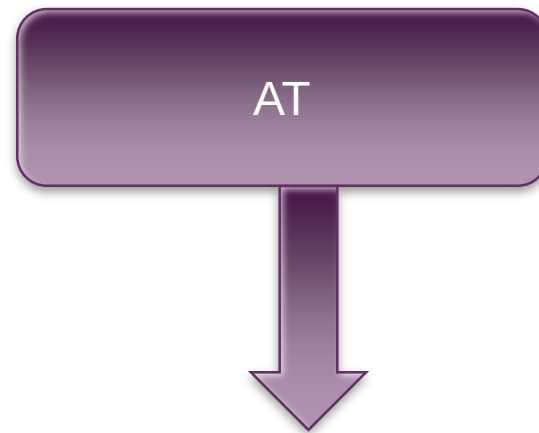
Explication :

Cette ligne de **crontab** signifie :

- `0` → à 0 minute
- `8` → à 8 heures du matin
- `* * *` → tous les jours, tous les mois, tous les jours de la semaine
- La commande `echo "Bonjour !" >> /home/student/bonjour.txt` sera exécutée à 8h00 chaque jour, ajoutant le message "Bonjour !" dans le fichier `bonjour.txt`.

Résultat attendu :

Un nouveau message "Bonjour !" sera ajouté chaque jour à 8h dans le fichier `/home/student/bonjour.txt`.



```
echo "echo Bonjour ! >> /home/student/bonjour.txt" | at now + 2 minutes
```

Explication :

- `echo "..."` : définit la commande à exécuter dans 2 minutes.
- `| at now + 2 minutes` : planifie l'exécution **une seule fois**, 2 minutes après l'heure actuelle.

Résultat attendu :

Après 2 minutes, le message "**Bonjour !**" sera ajouté une fois dans le fichier `/home/student/bonjour.txt`.



Pratique

+ Le cron

- ❑ Cron est un diminutif de crontab qui signifie chrono table ou table de planification en français
- ❑ Cron comprend un “démon” (dragon) **crond** et une commande d’édition
- ❑ Chaque utilisateur peut mettre en œuvre l’exécution automatique des tâches.
- ❑ On programme le lancement de commande avec la commande crontab en : Minutes, heures, jour du mois, mois et jour de la semaine



Daemon crond

10

Cron

/

\

/etc/crontab

anacron

/etc/cron.d/

/etc/cron.hourly ; /etc/cron.daily

crontab -e

/etc/cron.weekly; /etc/cron.monthly

- /etc/crontab exécute les jobs d'une manière régulière
- -crontab -e : exécute crontab pour un utilisateur spécifique
- anacron : s'occuper des jobs qui ont besoin d'être exécutés chaque heure, jour semaine ou mois :

+ Fichier /etc/crontab

```
[hayet@localhost ~]$ cat /etc/crontab
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
```

```
# For details see man 4 crontabs
```

```
# Example of job definition:
```

```
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR s
# | | | | |
# * * * * * user-name  command to be executed
```

+ Le cron

- ❑ Une ligne correspond à une tâche à exécuter en

mm hh jj MMM JJJ

- ❑ mm représente les minutes (de 0 à 59)
- ❑ hh représente l'heure (de 0 à 23)
- ❑ jj représente le numéro du jour du mois (de 1 à 31)
- ❑ MMM représente l'abréviation du nom du mois (jan, feb, ...) ou bien le numéro du mois (de 1 à 12)
 - JJJ représente l'abréviation du nom du jour ou bien le numéro du jour dans la semaine :
0 = Dimanche, 1 = Lundi, 2 = Mardi, ..., 6 = Samedi , 7 = Dimanche

+ Le cron

❑ Pour chaque valeur numérique (mm, hh, jj, MMM, JJJ)

les notations possibles sont :

- * : à chaque unité (0, 1, 2, 3, 4...)
- 5,8 : les unités 5 et 8
- 2-5 : les unités de 2 à 5 (2, 3, 4, 5)
- */3 : toutes les 3 unités (0, 3, 6, 9...)
- 10-20/3 : toutes les 3 unités (10, 13, 16, 19)

- Toutes les 5 minutes

```
*/5 * * * * df >> /tmp/df.log
```

+ Le cron

14

Commande pour exécuter backup.sh deux fois par jour (à 2h30 et 14h30) du lundi au vendredi
`30 2,14 * * 1-5 root /usr/local/sbin/backup.sh`

30	2,14	*	*	1-5	root	/usr/local/sbin/backup.sh
0-59	0-23	1-31	1-12	0-7	Commande à exécuter	
					Utilisateur qui exécute la commande	
				Jour de la semaine		
				Dimanche = 0 ou 7; lundi = 1; mardi = 2; mercredi = 3; jeudi = 4; vendredi = 5; samedi = 6		
			Mois			
			Janvier = 1 (...) Décembre = 12			
		Jour				
		Heure				
		Minute				

<http://jhroy.ca>

+ Le cron : short cuts

string	meaning	in 5 fields format
@reboot	Run once, <u>at startup</u> .	N/A
@yearly	Run once a year	0 0 1 1 *
@annually	(same as @yearly)	
@monthly	Run once a month	0 0 1 * *
@weekly	Run once a week	0 0 * * 0
@daily	Run once a day	0 0 * * *
@midnight	(same as @daily)	
@hourly	Run once an hour	0 * * * *
@every_minute	Run once a minute	*/1 * * * *
@every_second	Run once a second	

+ Le cron

- Lister les cron

```
#crontab -l
```

- Éditer la liste de cron

```
#crontab -e
```

- Supprimer les cron

```
#crontab -r
```

- Changer d'éditeur

```
#export EDITOR=VIM
```

- ❑ at permet d'indiquer l'heure du lancement de manière assez complexe
- ❑ at accepte des spécifications de la forme HH:MM pour exécuter un travail à une heure donnée de la journée en cours (ou du lendemain si l'heure mentionnée est déjà dépassée).
- ❑ On peut également demander à at de déclencher le travail le jour même en ajoutant le suffixe today ou le lendemain en ajoutant tomorrow

- ❑ On peut aussi lui fournir l'un des arguments suivants :
midnight (minuit), noon (midi), ou teatime (l'heure du thé, soit 16 heures).
- ❑ On peut indiquer le jour du lancement, en précisant une date du type nom-du- mois, jour année (en anglais).
- ❑ On peut indiquer une heure de la forme now (maintenant) + nombre d'unités, où les unités peuvent être minutes, hours (heures), days (jours), ou weeks (semaines)

+ AT

- ❑ Le super-utilisateur peut toujours employer at.
- ❑ Les permissions des utilisateurs sont déterminées par les fichiers `/etc/at.allow` et `/etc/at.deny`
- ❑ Si le fichier `/etc/at.allow` existe, seuls les utilisateurs dont les noms sont mentionnés dans ce fichier peuvent utiliser at.
- ❑ Si `/etc/at.allow` n'existe pas, at vérifie si `/etc/at.deny` existe, et tous les utilisateurs non-mentionnés dans ce fichier ont le droit d'invoquer at.
- ❑ Si aucun de ces deux fichiers n'existe, seul le super-utilisateur a le droit d'appeler at.
- ❑ Un fichier `/etc/at.deny` vide signifie que tous les utilisateurs ont le droit d'exécuter at

- ❑ `at` et `batch` lisent, depuis l'entrée standard, ou depuis un fichier, des commandes qu'elles exécuteront ultérieurement, en utilisant `/bin/sh`.
- ❑ `at` lance les commandes indiquées à une heure précise.
- ❑ `atq` affiche la liste des commandes en attente d'exécution, sauf si l'appelant est le Super-utilisateur.
- ❑ `atrm` supprime les travaux en attente.

+ Utilisation de la commande AT

- Lancer une commande a une midi

```
#at 12:00
at> echo "j ecris dans un fichier" >/tmp/at.test
at> <EOT>
job 1 at Fri Nov 13 12:00:00 2015
```

- Lancer une commande demain à 11h54

```
#at 11:54 tomorrow
at> echo "j ecris dans un fichier" >/tmp/at.test
at> <EOT>
job 2 at Sat Nov 14 11:54:00 2015
```

- Lancer une commande dans 5 minutes

```
#at now +5 minutes
```

+ Utilisation de la commande AT

- Lancer une commande a une date précise à 12h:

```
#at 12:00 19.05.2020
```

```
at> echo "j écris dans un fichier" >/tmp/at.test
```

```
at> ctrl+d
```

+ Utilisation de la commande AT

- Lister les travaux a exécuter

```
## atq
```

```
2    Sat Nov 14 11:54:00 2015 a root
3    Fri Nov 13 12:03:00 2015 a root
4    Tue Nov 17 11:59:00 2015 a root
5    Fri Nov 27 11:59:00 2015 a root
```

- Supprimer un job

```
# atrm 3
```

```
#atq
```

```
2    Sat Nov 14 11:54:00 2015 a root
4    Tue Nov 17 11:59:00 2015 a root
5    Fri Nov 27 11:59:00 2015 a root
```